

VITyarthi -- Build Your Own Project

Student Result Management System

A CLI-Based Python Application for Managing Student Marks, Grading, and Class Analytics

Course Project Report

Subject: Python Programming (Modules 1–12)

Submitted as part of the flipped-course evaluation

1. Introduction

Tracking student marks manually or in scattered spreadsheets is time-consuming and error-prone, and makes it difficult to quickly answer class-level questions such as “who is the topper?” or “what is the pass percentage in a subject?”. The Student Result Management System (SRMS) is a command-line Python application, built entirely with the Python standard library, that lets a teacher or administrator maintain a roster of students, record subject-wise marks, automatically compute grades according to a configurable policy, and generate class-wide analytics -- all without needing a database server, a GUI framework, or an internet connection.

The project was designed to apply, in a single cohesive codebase, every concept taught across the course syllabus: variables and operators, input/output, operator precedence, type conversion, the core data structures (list, tuple, dict, set), control flow, functions, modules & packages, array-based data storage, and full object-oriented programming.

2. Problem Statement

Design and implement a fully offline, command-line result management system that allows a teacher to: (a) maintain a roster of students; (b) record and update marks for each student across a fixed set of subjects, with robust input validation; (c) automatically compute each student's total, percentage, and letter grade using a transparent and consistent policy; and (d) generate class-wide insights -- a rank list, the topper, subject-wise averages, and a pass/fail summary -- and export them for further use. The solution must be executable purely from the terminal, persist data between runs, and be organised into clean, testable, modular code.

3. Functional Requirements

The system is organised into three major functional modules, each with a clear input/output structure and a logical workflow (see Section 7, Use Case Diagram, and Section 4, Design & Documentation):

3.1 Student & Marks Management

- Add a new student (unique roll number + name), with duplicate detection.
- Enter or update a student's marks, subject by subject, with input validation.
- Search students by (partial, case-insensitive) name.
- List all students with their current percentage and grade.
- Remove a student from the roster, with an explicit confirmation step.

3.2 Result Processing & Grading

- Compute total marks and percentage from a student's per-subject marks.
- Map percentage to a letter grade (A+ through F) via a configurable grade table.
- Determine per-subject pass/fail and overall pass/fail (aggregate threshold AND no individual subject failures).
- Generate a formatted, human-readable marksheet for any student.

3.3 Reporting & Analytics

- Rank the entire class by percentage (with roll-number tie-breaking).
- Identify the class topper and compute the class average.
- Compute subject-wise class averages.
- Compute a pass/fail summary for the class.
- Export a full class report (rank, roll no, name, total, percentage, grade, result) to CSV.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	All operations are $O(n)$ or better over the in-memory student list (a handful of students to a few thousand);

	the roster is loaded once at startup and only re-read/written on explicit save, avoiding repeated disk I/O.
Reliability	Every user-facing action is wrapped in a single top-level exception handler in main.py that catches the entire SRMSError hierarchy, so malformed input or a missing record never crashes the program.
Usability	A numbered menu, clear prompts, aligned tabular output, and explicit confirmation before destructive actions (student removal) keep the CLI approachable for a non-technical user.
Maintainability	A layered architecture (CLI -> Service -> Utility -> Model -> Data Access) with single-responsibility modules means a change to, e.g., the CSV format only touches file_handler.py.
Error-Handling Strategy	A custom exception hierarchy (SRMSError and 7 subclasses) gives every failure mode a specific, catchable type and a friendly message, instead of raw Python tracebacks.
Resource Efficiency	The `array` module is used for class-wide percentage figures (a homogeneous numeric dataset), which is more memory-compact than a general-purpose list of Python floats.

5. System Architecture

SRMS follows a layered architecture. The CLI (main.py) is the only layer the user interacts with; it delegates all business logic to the Service layer (ResultManager for CRUD, ReportService for grading/analytics), which in turn relies on the Utility layer for validation and grade computation, the Model layer (Student, Subject) for encapsulated OOP behaviour, and the Data Access layer (FileHandler) for all CSV I/O. Each layer depends only on the layer(s) directly below it, which keeps the codebase easy to test, extend, and reason about.

System Architecture -- Student Result Management System

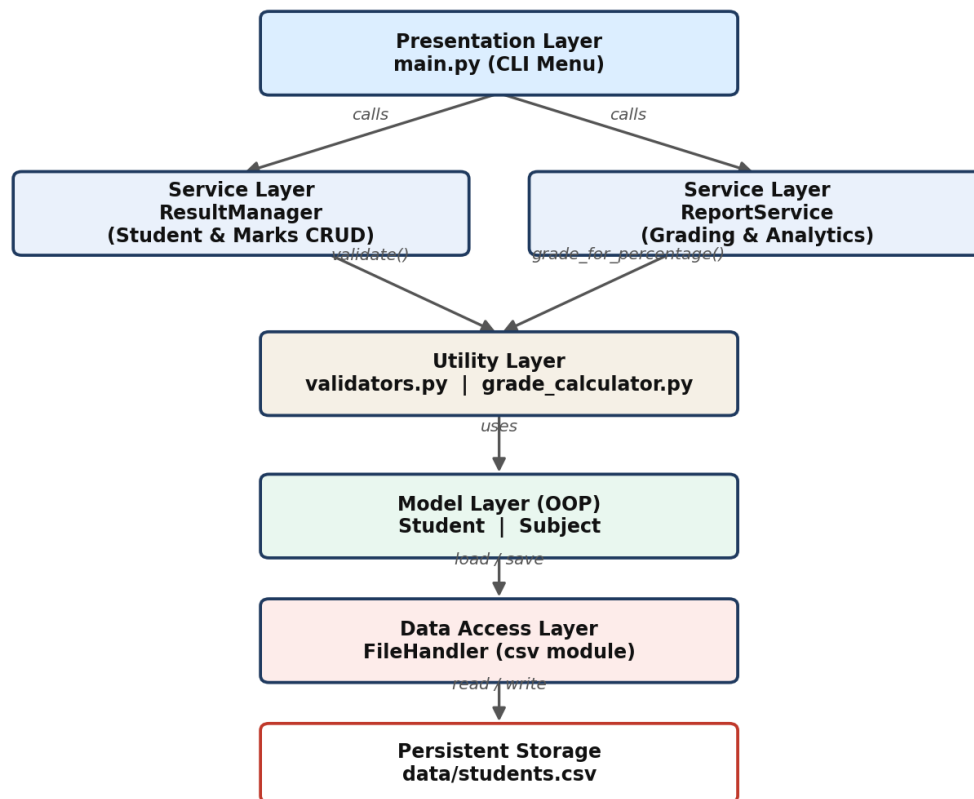


Figure 1: System Architecture Diagram

6. Process Flow / Workflow

The application runs a continuous menu loop: load data, display the menu, read and validate the user's choice and input, execute the corresponding action, display the result, and repeat until the user chooses to save and exit. Invalid input at any point is caught and reported without disrupting the loop.

Process Flow -- Main Program Workflow

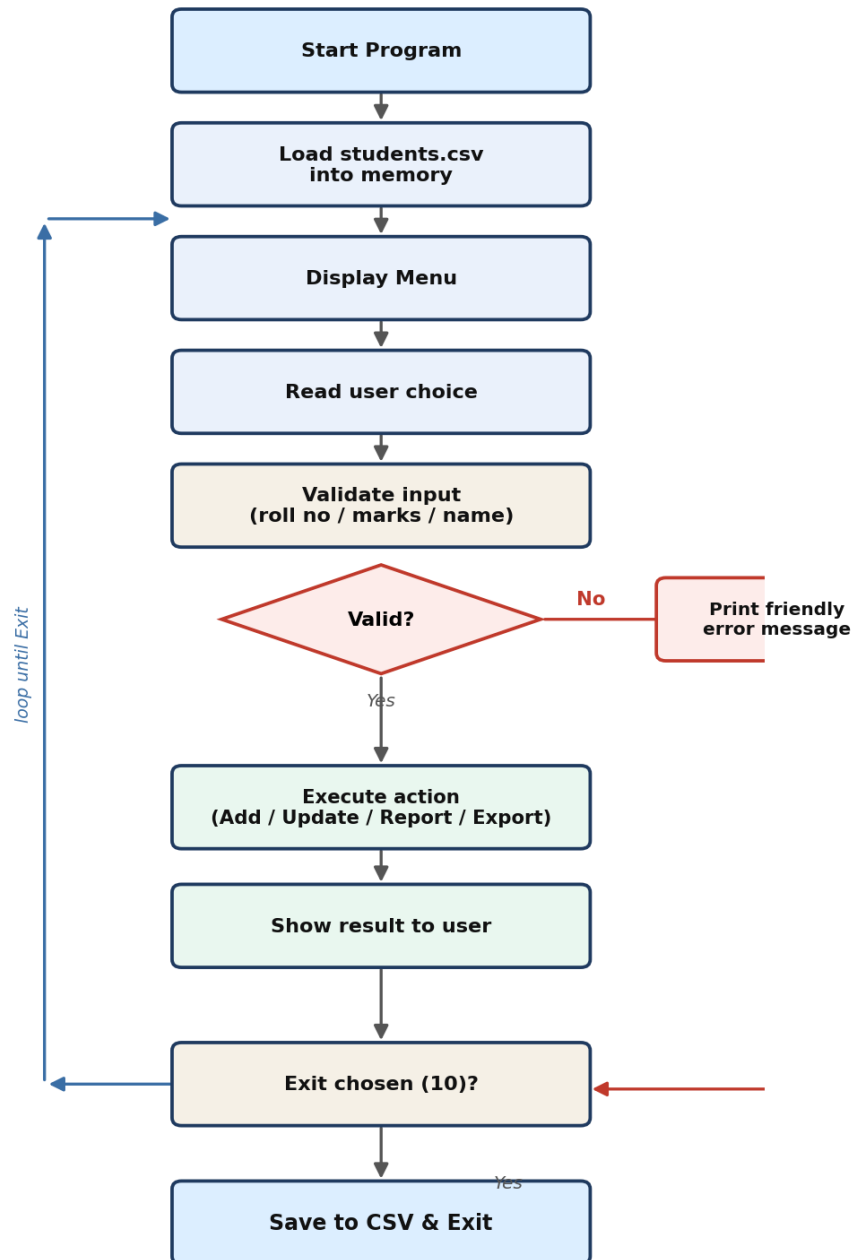


Figure 2: Process Flow / Workflow Diagram

7. Use Case Diagram

A single actor -- the Administrator/Teacher -- interacts with all nine use cases exposed by the system, corresponding one-to-one with the main menu options.

Use Case Diagram -- SRMS

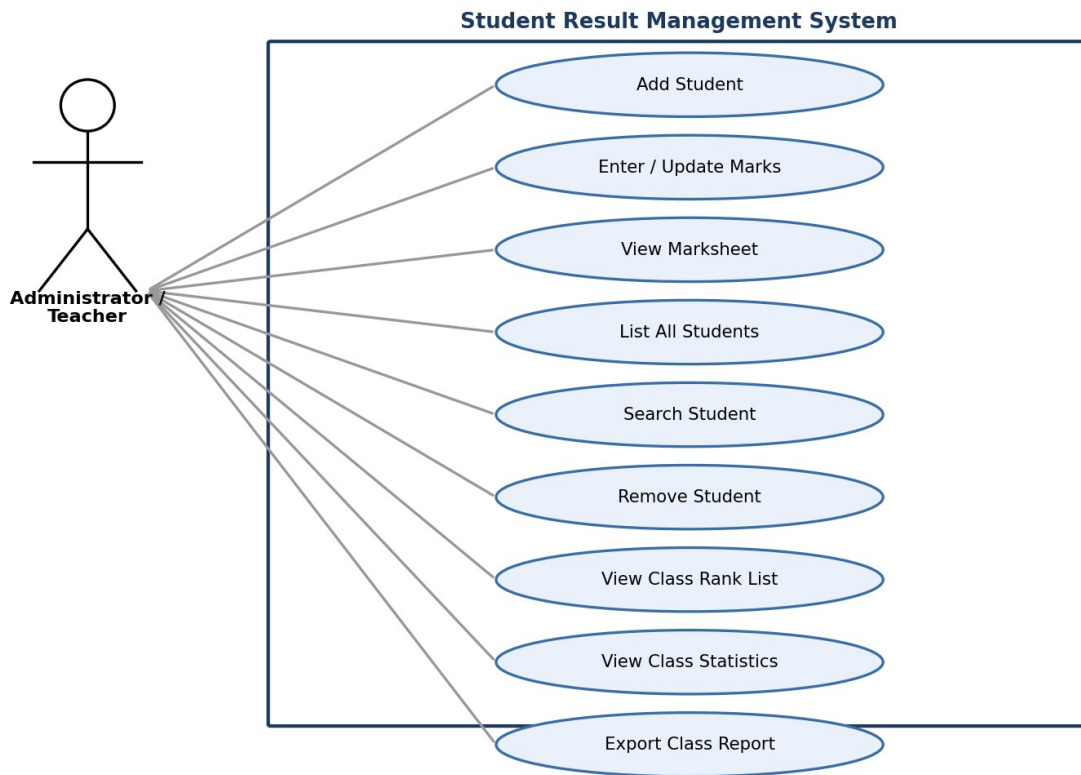


Figure 3: Use Case Diagram

8. Class / Component Diagram

Student is the central OOP class, composed of Subject objects and providing derived properties (total, percentage, grade, pass/fail). ResultManager owns the in-memory roster and CRUD operations; ReportService is a stateless collection of grading/analytics operations; FileHandler isolates all CSV I/O; and every business-rule failure raises a subclass of the common SRMSError base exception.

Class Diagram -- SRMS (core classes)

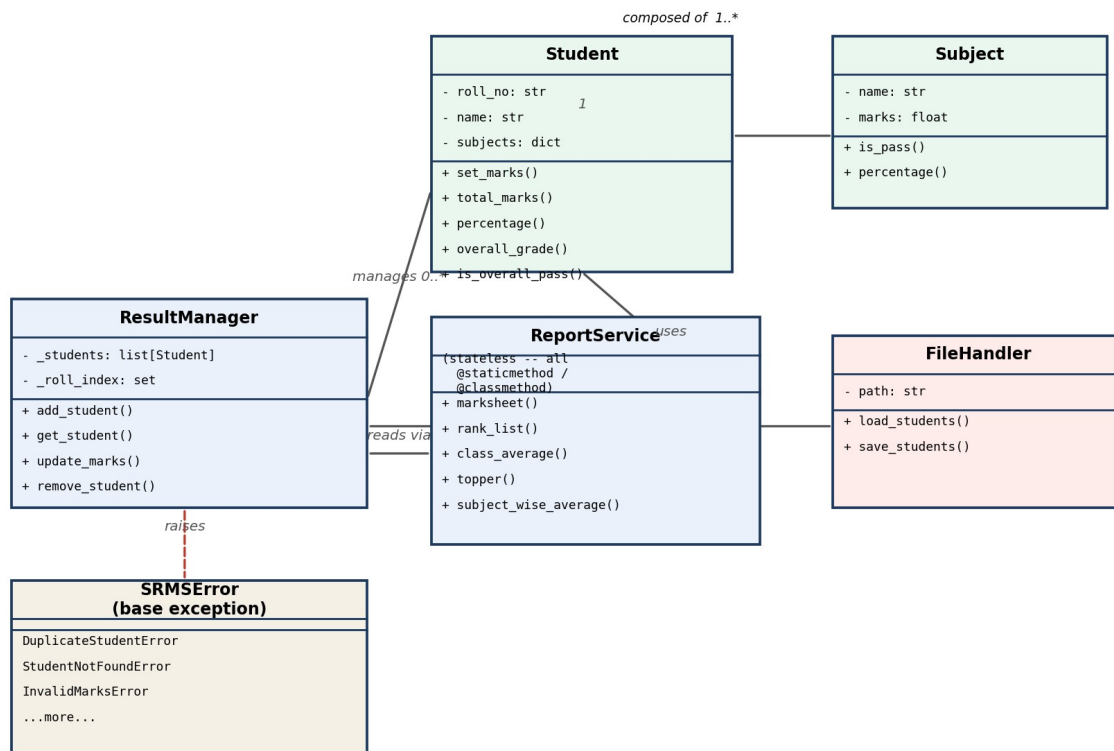


Figure 4: Class Diagram

9. Sequence Diagram

The sequence below traces the “Enter / Update Marks” use case end-to-end: the CLI first confirms the student exists, validates the raw marks input via the Utility layer, then asks **ResultManager** to update the **Student** object's marks before confirming success back to the user.

Sequence Diagram -- "Enter / Update Marks" Flow

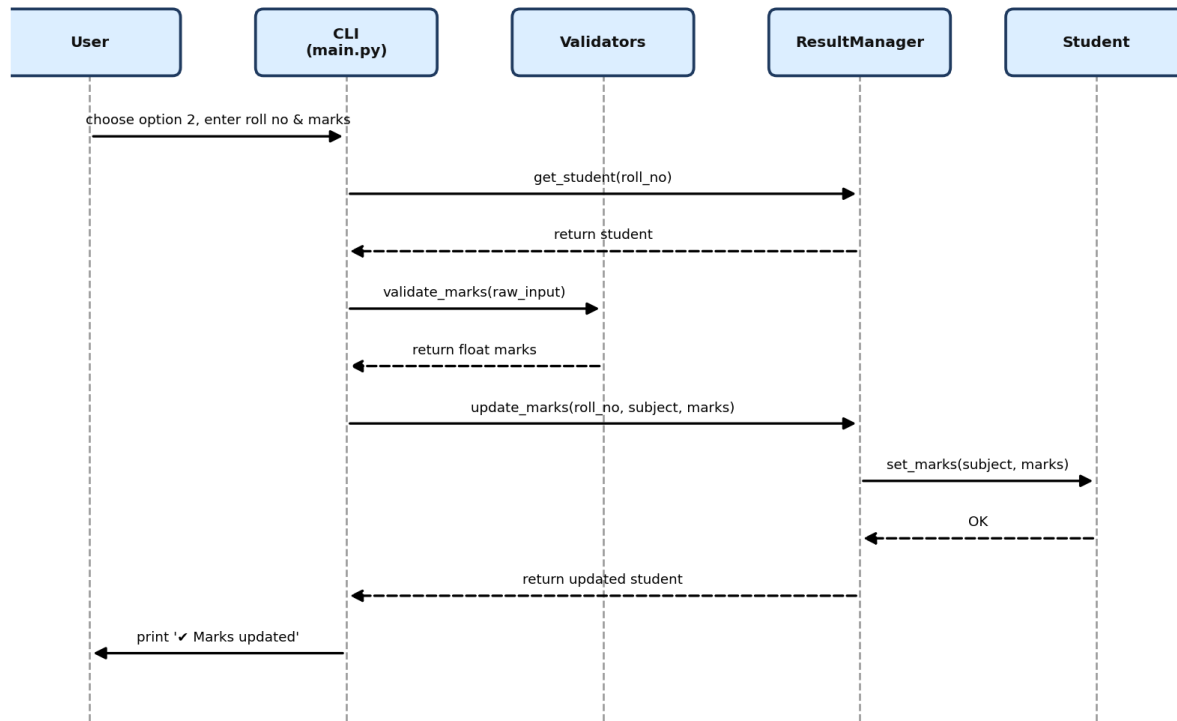


Figure 5: Sequence Diagram -- Enter / Update Marks

10. Data / Storage Design

Data is persisted as a flat CSV file (data/students.csv) with one row per student. This was chosen deliberately over a database because (a) the course syllabus's scope is core Python rather than database technology, and (b) it keeps the project runnable anywhere with zero setup. The schema is:

Column	Type	Description
roll_no	string (unique)	Primary key -- alphanumeric student identifier
name	string	Student's full name
Python	float (0-100)	Marks scored in Python
Mathematics	float (0-100)	Marks scored in Mathematics
Data Structures	float (0-100)	Marks scored in Data Structures
English	float (0-100)	Marks scored in English
Computer Networks	float (0-100)	Marks scored in Computer Networks

11. Design Decisions & Rationale

- Layered architecture: chosen so that grading logic, validation, persistence and presentation can each change independently -- e.g. swapping CSV for SQLite would only require editing FileHandler.
- Custom exception hierarchy over generic exceptions: gives calling code (main.py) a single catch-all (SRMSError) while still letting each failure carry a specific, descriptive type.
- Dict-of-Subject-objects on Student (rather than a flat list of marks): gives O(1) subject lookup/update and keeps each mark paired with pass/fail behaviour via the Subject class.
- Set for roll-number indexing in ResultManager: turns duplicate-detection from an O(n) scan into an O(1) lookup.

- array module for class-wide percentages: a deliberate, syllabus-aligned use of Python's built-in array data structure for homogeneous numeric data, as distinct from a general-purpose list.
- Class/static methods on ReportService: the service is stateless by design (it only derives facts from the students handed to it), so its methods don't need an instance.

12. Implementation Details

The codebase spans 13 Python files organised into a proper package structure (src/, src/models/, src/services/, src/utils/, tests/), exceeding the minimum of 5–10 meaningful modules. Key implementation points:

- Type conversion: raw CSV values and raw console input (always strings) are explicitly converted to float/int in validators.py and Student.from_csv_row(), with ValueError caught and re-raised as InvalidMarksError.
- Control flow: the CLI's main loop, the grade-boundary lookup in grade_calculator.py, and every validation function rely on if/elif/for/while constructs taught in Modules 7–8.
- Functions & modules/packages: business logic is decomposed into small, single-purpose functions and organised into importable packages (src.models, src.services, src.utils), directly applying Modules 9–10.
- Data structures: list (roster), dict (a student's subjects), tuple (immutable SUBJECTS and GRADE_TABLE), set (roll-number index), and array (class percentages) are each used where they are the **appropriate** structure, not interchangeably.
- OOP: Student and Subject are full classes with encapsulated state and behaviour; Student defines __str__, __repr__, __eq__ and __hash__; classmethod from_csv_row() acts as an alternate constructor; the exception hierarchy demonstrates inheritance.
- Error handling: a single try/except SRMSError block in the CLI's main loop converts every business-rule violation into a friendly, non-crashing message.

13. Screenshots / Results

Sample runs of the application, captured directly from the terminal:

```
• • • terminal -- python3 main.py

Student Result Management System -- type Ctrl+C at any time to abort safely.

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10):
Roll No    Name                %    Grade
-----
21BCE001   Aarav Sharma         79.00 B+
21BCE002   Diya Patel           90.60 A+
21BCE003   Kabir Singh          49.60 D
21BCE004   Meera Nair           64.20 B
21BCE005   Rohan Gupta          35.00 F

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10): Data saved. Goodbye!
```

Figure 6: Main menu and student list

```

• • • terminal -- view marksheet

Student Result Management System -- type Ctrl+C at any time to abort safely.

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10): Roll number: =====
MARKSHEET -- Diya Patel (Roll No: 21BCE002)
=====
Python      : 95.00 / 100 [Pass]
Mathematics : 91.00 / 100 [Pass]
Data Structures : 89.00 / 100 [Pass]
English     : 85.00 / 100 [Pass]
Computer Networks : 93.00 / 100 [Pass]
-----
Total       : 453.00 / 500
Percentage  : 90.60%
Grade      : A+
Result     : PASS
=====

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10): Data saved. Goodbye!

```

Figure 7: Individual student marksheet

```

• • • terminal -- class statistics

Student Result Management System -- type Ctrl+C at any time to abort safely.

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10):
Class average      : 63.68%
Topper             : Diya Patel (21BCE002) -- 90.6%
Passed / Failed   : 4 / 1 (of 5)
Subject-wise average:
  Python           : 65.0
  Mathematics      : 60.4
  Data Structures  : 64.8
  English          : 64.8
  Computer Networks : 63.4

STUDENT RESULT MANAGEMENT SYSTEM
1. Add new student
2. Enter / update marks for a student
3. View a student's marksheet
4. List all students
5. Search students by name
6. Remove a student
7. Class rank list
8. Class statistics (average, pass/fail, topper)
9. Export class report to CSV
10. Save & Exit

Choose an option (1-10): Data saved. Goodbye!

```

Figure 8: Class-wide statistics

14. Testing Approach

The project includes an automated unit test suite (tests/test_grade_calculator.py, 17 tests) built with Python's unittest framework, covering:

- Grade-boundary computation at every threshold edge (e.g. 89.99% -> 'A', 90% -> 'A+').
- Validator functions: valid and invalid roll numbers, names, marks (non-numeric, out of range), and subjects.
- The Student model: total/percentage arithmetic, overall grade, pass/fail with and without a failed subject, and equality-by-roll-number.

In addition, the full CLI was manually exercised end-to-end (adding a student, entering marks, viewing a marksheet, listing, ranking, computing statistics, exporting, and saving) and specific error paths were verified interactively -- duplicate roll numbers, non-existent roll numbers, and non-numeric marks input all produce a friendly error message rather than a crash. Run the suite with:

```
python3 -m unittest discover -s tests -v
```

15. Challenges Faced

- Deciding the overall pass/fail rule: settled on requiring both a minimum aggregate percentage AND no individual subject failures, matching how most real academic systems compute results, rather than a single aggregate-only check.
- Keeping the CSV read/write layer perfectly in sync with the Student model's fields, solved by defining CSV_FIELDNAMES once in config.py and having both the reader (Student.from_csv_row) and writer (Student.to_csv_row) depend on the same SUBJECTS tuple.
- Producing clear, exam-ready design diagrams without relying on an external diagramming tool or the internet -- solved by writing a small, reusable matplotlib-based diagram generator (docs/generate_diagrams.py) so every diagram is reproducible from source.

16. Learnings & Key Takeaways

- How to structure a non-trivial Python project into clean, single-responsibility packages rather than one large script.
- Why choosing the right data structure (set for membership checks, array for homogeneous numeric data, tuple for fixed configuration) matters even in a small project.
- How a custom exception hierarchy simplifies error handling at the call site while preserving specific, debuggable failure information.
- The value of writing automated tests alongside the implementation, which caught an early off-by-one error in the grade-boundary logic.

17. Future Enhancements

- Swap the CSV backend for SQLite behind the same FileHandler interface to support much larger datasets.
- Add bulk CSV import for onboarding an entire class at once.
- Generate a per-student PDF marksheet (reusing the pdf-generation approach used for this report).
- Wrap the existing service layer in a small Flask web front-end without changing any business logic.

18. References

- Python Software Foundation -- Python 3 Standard Library Documentation: <https://docs.python.org/3/library/>
- Python Software Foundation -- csv module documentation: <https://docs.python.org/3/library/csv.html>
- Python Software Foundation -- array module documentation: <https://docs.python.org/3/library/array.html>
- Python Software Foundation -- unittest module documentation: <https://docs.python.org/3/library/unittest.html>
- Course modules 1–12 (Introduction to Python through Object Oriented Programming in Python), as supplied on the course platform.